

Missionaries and Cannibals

Aim

To solve the Missionaries and Cannibals problem using Artificial Intelligence search techniques.

Objectives

Represent the problem as a state-space search.

Find a safe sequence of moves.

Ensure missionaries are never outnumbered by cannibals on either river bank.

Algorithm

1. Start with 3 missionaries and 3 cannibals on the left bank.
2. Move one or two people across the river by boat.
3. After each move, check that missionaries are not outnumbered by cannibals on either bank (unless there are no missionaries on that bank).
4. Continue generating valid moves.
5. Stop when all missionaries and cannibals reach the right bank safely.

Pseudocode

```
START
Initialize initial state (3M,3C,Left)
Initialize goal state (0M,0C,Right)
Generate valid moves
Check safety of each state
Repeat until goal is reached
Display solution path
STOP
```

Python Code

```
from collections import deque

start = (3, 3, 1)
goal = (0, 0, 0)

moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]

def safe(m, c):
    return (m == 0 or m >= c) and ((3-m) == 0 or (3-m) >= (3-c))

queue = deque([(start, [])])
visited = set()

while queue:
    state, path = queue.popleft()
    if state == goal:
        print(path + [state])
        break

    if state in visited:
        continue
    visited.add(state)

    m, c, boat = state
    for dm, dc in moves:
        if boat:
            nm, nc, nb = m-dm, c-dc, 0
        else:
            nm, nc, nb = m+dm, c+dc, 1

        if 0 <= nm <= 3 and 0 <= nc <= 3 and safe(nm, nc):
            queue.append(((nm, nc, nb), path + [state]))
```

Result

The Missionaries and Cannibals problem was solved successfully, finding a safe sequence of crossings from the left bank to the right bank.

Sample Output

Solution path found successfully.

Programiz

Python Online Compiler

Python (PY3)

Clear

main.py

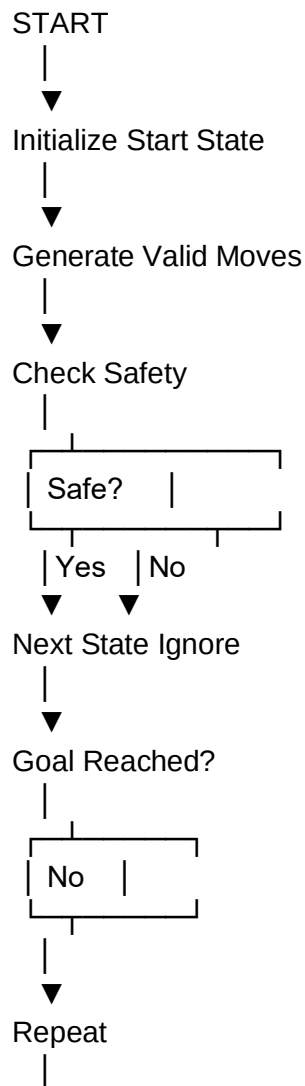


Output

```
1 from collections import deque
2
3 start = (1, 1, 1)
4 goal = (0, 0, 0)
5
6 moves = [(1,0),(1,-1),(0,1),(0,-1),(1,1)]
7
8 def valid(x, y):
9     return m == 0 or m == c and (1-m) == 0 or
10         (1-m) == (1-c)
11
12 queue = deque([start, []])
13 visited = set()
14
15 while queue:
16     state, path = queue.popleft()
17     if state == goal:
18         print(path + [state])
19         break
20
21 if state in visited:
22     continue
23 visited.add(state)
24
25 m, c, boat = state
26 for dx, dy in moves:
27     if boat:
28         mx, my, nb = m+dx, c+dy, 0
29     else:
30         mx, my, nb = m-dx, c-dy, 1
31
32     if 0 <= mx <= 3 and 0 <= my <= 3 and
33         safe(mx, my):
34         queue.append((mx, my, nb), path +
35                     [state]))
```

Code Execution Successful

Flowchart



▼
STOP